

Kiste Phase 6 Spec

Many Repo Providers + Docker Hub + Serverless Target Planning

One-sentence spec: Kiste Phase 6 expands Kiste to scan many repo providers, inspect Docker Hub and container image repositories, link source repos to images, and recommend whether each service should target Kubernetes or serverless/container-serverless platforms.

1. Phase 6 Position

Phase 6 should come after Phase 5 because provider expansion becomes much cleaner once Kiste has a stable Python library and a CLI wrapper. The point of this phase is not to deploy everything yet, but to make Kiste understand more places where code and containers live, then generate a target plan for Kubernetes or serverless/container-serverless execution.

Phase	Focus
1	Local Docker repo scan + token inspection
2	GitHub/Hugging Face remote scan + CI testing
3	Multi-repo workspace + token matrix
4	GitOps repo + SOPS encrypted secrets
4.5	Review and architecture cleanup
5	Python library + CLI
6	Many repo providers + Docker Hub + serverless target recommendation

2. Main Features

- Many source repo providers: GitHub, GitLab, Bitbucket, Azure DevOps Repos, Hugging Face Hub, Generic Git URL, and self-hosted Git servers.
- Docker Hub and container image repo inspection: tags, digest, architecture, exposed ports, entrypoint, environment hints, and pull permission.
- Serverless and container-serverless target planning: decide whether each service fits Kubernetes, AWS Lambda container image, Google Cloud Run, or Azure Container Apps.

Kiste should now understand the chain from source code to container image to deployment target.

```
source code repo
-> builds
container image repo
-> deploys to
Kubernetes or serverless
```

3. Source Repo Providers

Phase 6 extends the provider system from the earlier GitHub/Hugging Face model into a broader adapter-based repository layer.

```
RepoProvider
|-- GitHubProvider
|-- GitLabProvider
|-- BitbucketProvider
|-- AzureDevOpsProvider
|-- HuggingFaceProvider
```

```
|-- GenericGitProvider
```

All providers should normalize into the same internal repo model:

```
{
  "name": "api",
  "provider": "github",
  "repo_ref": "khoa/robot-api",
  "default_branch": "main",
  "scan_mode": "api",
  "token": "github-read-dev"
}
```

4. Docker Hub and Container Image Repo Support

Phase 6 starts with Docker Hub image inspection. Later phases can add GHCR, GitLab Container Registry, AWS ECR, Google Artifact Registry, Azure Container Registry, Quay, Harbor, and generic OCI registry support.

Core image commands:

```
kiste image inspect dockerhub khoa/robot-api
kiste image inspect docker.io/khoa/robot-api:latest
kiste image tags docker.io/khoa/robot-api
kiste image check-pull docker.io/khoa/robot-api:latest
```

Kiste should detect:

- image name, registry provider, repository, tag, and digest
- image size, created time, operating system, and architecture
- exposed ports, entrypoint, command, labels, and environment variable names
- pull permission and tag freshness
- whether the image uses a mutable tag such as latest

Example output:

Image: docker.io/khoa/robot-api:latest

```
[OK] Registry: Docker Hub
[OK] Image exists
[OK] Tag: latest
[OK] Architecture: linux/amd64
[OK] Exposed port: 8000
[WARN] Uses mutable tag latest
[WARN] Digest pin recommended
```

5. Source-to-Image Link

Deployment uses container images, not source code directly. Kiste should connect each source repo to the image it produces.

```
repos:
- name: api
  provider: github
  repo: khoa/robot-api
  type: backend
  image: docker.io/khoa/robot-api:latest

- name: worker
  provider: gitlab
  repo: robot/worker
  type: worker
  image: docker.io/khoa/robot-worker:latest
```

New command:

kiste link image api `docker.io/khoa/robot-api:latest`

Graph output:

api source repo -> `docker.io/khoa/robot-api:latest` -> deployment target
worker source repo -> `docker.io/khoa/robot-worker:latest` -> deployment target

6. Serverless Target Planning

Phase 6 should recommend a target. It should not fully deploy yet. Kiste should decide whether a service is a better fit for Kubernetes or for a serverless/container-serverless platform.

Supported target types:

- Kubernetes
- AWS Lambda container image
- Google Cloud Run
- Azure Container Apps

7. Target Recommendation Rules

Target fit	Signals
Serverless / container-serverless	Stateless HTTP API; small web service; low or variable traffic; event-driven worker; simple container image; no GPU; no long-running socket; no local persistent storage.
Kubernetes	Stateful service; stable internal networking; GPU; long-running worker; sidecars; custom networking; service mesh; persistent volume; highly connected microservice graph.

Example target plan:

```
api:
  recommended_target: cloud-run
  reason: stateless HTTP service, port 8000, no GPU

worker:
  recommended_target: kubernetes
  reason: long-running background worker, queue dependency

model-inference:
  recommended_target: kubernetes
  reason: GPU likely required
```

8. New Commands

Provider commands:

```
kiste provider list
kiste provider add github
kiste provider add gitlab
kiste provider add bitbucket
kiste provider add azure-devops
kiste provider add hf
```

Repo scan commands:

```
kiste scan github owner/repo
kiste scan gitlab group/project
kiste scan bitbucket workspace/repo
kiste scan azure-devops org/project/repo
kiste scan hf owner/model
kiste scan git https://example.com/team/repo.git
```

Image commands:

```
kiste image inspect docker.io/khoa/robot-api:latest
kiste image tags docker.io/khoa/robot-api
kiste image check-pull docker.io/khoa/robot-api:latest
```

Target planning commands:

```
kiste target plan
kiste target plan --service api
kiste target plan --prefer serverless
kiste target plan --prefer kubernetes
kiste workspace check --include-images --target-plan
```

9. Phase 6 Architecture

```
Kiste Python Library + CLI
|
v
Provider Registry
|-- Source Repo Providers
|   |-- GitHub
|   |-- GitLab
|   |-- Bitbucket
|   |-- Azure DevOps
|   |-- Hugging Face
|   |-- Generic Git
|
|-- Image Registry Providers
|   |-- Docker Hub
|   |-- Generic OCI later
|
|-- Deployment Target Providers
|   |-- Kubernetes
|   |-- AWS Lambda Container
|   |-- Google Cloud Run
|   |-- Azure Container Apps
|
v
Normalized Models
|-- RepoSpec
|-- ImageSpec
|-- ServiceSpec
|-- TargetPlan
|-- WorkspaceSpec
|
v
Report + GitOps Generator later
```

10. Data Models

ImageSpec:

```
{
  "image": "docker.io/khoa/robot-api:latest",
  "registry": "dockerhub",
  "repository": "khoa/robot-api",
  "tag": "latest",
  "digest": "sha256:...",
  "architecture": ["linux/amd64"],
  "exposed_ports": [8000],
  "entrypoint": "uvicorn main:app",
  "warnings": ["mutable tag latest"]
}
```

TargetPlan:

```
{
  "service": "api",
  "recommended_target": "cloud-run",
  "alternatives": ["kubernetes", "azure-container-apps"],
  "reasons": [
    "stateless HTTP service",
    "container image available",
  ]
}
```

```

    "no GPU detected"
  ],
  "warnings": ["requires registry pull permission"]
}

```

11. What Phase 6 Should Not Do Yet

- No full production serverless deployment yet.
- No full multi-cloud provisioning.
- No full registry vulnerability scanner.
- No automatic cloud account creation.
- No full cost optimizer.
- No GPU scheduler.
- No dynamic secret runtime system.

Phase 6 should mainly handle repo provider expansion, Docker Hub image inspection, source-to-image linking, and serverless/Kubernetes target recommendation.

12. Acceptance Criteria

Phase 6 is complete when these commands work:

```

kiste scan gitlab group/project
kiste scan bitbucket workspace/repo
kiste scan azure-devops org/project/repo
kiste scan git https://example.com/team/repo.git

kiste image inspect docker.io/khoa/robot-api:latest
kiste image tags docker.io/khoa/robot-api
kiste image check-pull docker.io/khoa/robot-api:latest

kiste link image api docker.io/khoa/robot-api:latest
kiste target plan --service api
kiste workspace check --include-images --target-plan

```

Kiste should produce:

- [OK] repo provider abstraction works across many providers
- [OK] Docker Hub image metadata report
- [OK] image pull check
- [OK] source repo to image link
- [OK] Kubernetes vs serverless target recommendation
- [OK] workspace report including repo, image, and target plan

13. References

- Docker Hub repositories: <https://docs.docker.com/docker-hub/repos/>
- AWS Lambda container images: <https://docs.aws.amazon.com/lambda/latest/dg/images-create.html>
- Google Cloud Run: <https://cloud.google.com/run>
- Azure Container Apps: <https://azure.microsoft.com/en-us/products/container-apps>